

Plataforma Multioperador de Distribución de Última Milla

Entrega de análisis y diseño

Taller de Sistemas de Información .NET
Tecnólogo en Informática — UTEC

Equipo

Juliana Méndez	•	Martina Castro
Matías Alfaro	•	Cecilia Méndez

Docente

Nicolás Escobar Di Camillo

Índice

1	Arquitectura lógica	2
1.1	Módulos de negocio	2
1.2	Regla de dependencias	2
1.3	Aplicaciones desplegadas	4
1.4	Comunicación entre módulos	4
2	Arquitectura de despliegue previsto	5
2.1	Contenedores y réplicas	5
2.2	Continuidad ante la caída de una instancia	6
2.3	Nube	6
3	Modelo de dominio preliminar	7
3.1	Áreas y entidades	7
3.2	Relaciones que sostienen el diseño	7
3.3	Dónde entra el aislamiento entre inquilinos	7
3.4	Pendientes del modelo preliminar	8
4	Máquina de estados del envío	10
4.1	Transiciones válidas	10
4.2	Notas de diseño	11
5	Estrategia de multitenancy	13
5.1	Resolución del inquilino en tiempo de ejecución	13
5.2	Aislamiento en lecturas y escrituras	13
5.3	Pruebas automatizadas de aislamiento	13
5.4	Diferenciación por inquilino	13
6	Stack tecnológico y entorno de desarrollo	14
6.1	Stack confirmado	14
6.2	Opcionales elegidos (12 puntos)	15
6.3	Traefik como único proxy inverso	16
6.4	Observabilidad: dos backends, una sola instrumentación	16
6.5	Entorno de desarrollo	16
6.6	Trazabilidad contra los requerimientos no funcionales	17
7	Plan de trabajo	19
7.1	Asignación de responsabilidades	19
7.2	Cronograma por monitoreo	19
7.3	Prácticas del equipo	20
8	ADR-001 — Estilo arquitectónico y organización del código	21
9	ADR-002 — Estrategia de multitenancy	23

Esta entrega cubre los siete puntos pedidos para la instancia de análisis y diseño: los diagramas de arquitectura lógica y de despliegue previsto, el modelo de dominio preliminar, la máquina de estados del envío, la estrategia de multitenancy con su justificación, la propuesta de stack tecnológico y entorno de desarrollo, el plan de trabajo con responsables, y los dos primeros registros de decisión de arquitectura.

1. Arquitectura lógica

La solución es un monolito modular (API y aplicaciones web) más un Worker independiente que se comunica solo por la cola. Internamente sigue Clean Architecture con Vertical Slice: un proyecto por capa (Domain, Application, Infraestructure y presentación), con los casos de uso agrupados por módulo y por feature. La justificación está en el ADR-001 (sección 8). Una sola API autoriza; el Worker trabaja únicamente desde la cola.

Leyenda

Elemento	Significado
Rectángulo	Aplicación desplegable o módulo de negocio
Cilindro	Almacenamiento
Rectángulo doble	Broker de mensajes
Estadio punteado	Sistema externo a la plataforma
Flecha llena	Llamada directa
Flecha punteada	Salida asíncrona o telemetría

1.1 Módulos de negocio

Cada módulo es una carpeta dentro de Domain y de Application. Hacia afuera solo expone sus Commands, Queries y DTOs; ningún otro módulo usa sus entidades ni sus repositorios directamente.

Módulo	Responsabilidad	RF
Tenancy e Identidad	Operadores, comercios, usuarios, perfiles, resolución del inquilino, marca	1, 5
Configuración	Zonas, franjas, tarifarios y reglas operativas versionados	2, 3, 4, 6
Envíos	Alta, importación idempotente, bultos, tarifa, máquina de estados, eventos inmutables	7–12
Planificación	Hojas de ruta, validación de restricciones, orden de paradas, despacho	13–17
Ejecución y Sincronización	Descarga de ruta, escaneo, pruebas de entrega, posición, sincronización diferida	18–24
Seguimiento	Consulta pública por token, reprogramación	25, 26
Avisos y Notificaciones	Notificaciones al destinatario, webhooks firmados a comercios, historial y reenvío	27, 28
Operación y Reportes	Tablero en vivo, envíos en riesgo, reportes y liquidaciones	29, 30

1.2 Regla de dependencias

$\text{Domain} \leftarrow \text{Application} \leftarrow \text{Infraestructure}$. Domain no depende de ningún proyecto. Application depende solo de Domain y define las interfaces que necesita: `IUnitOfWork`, repositorios,

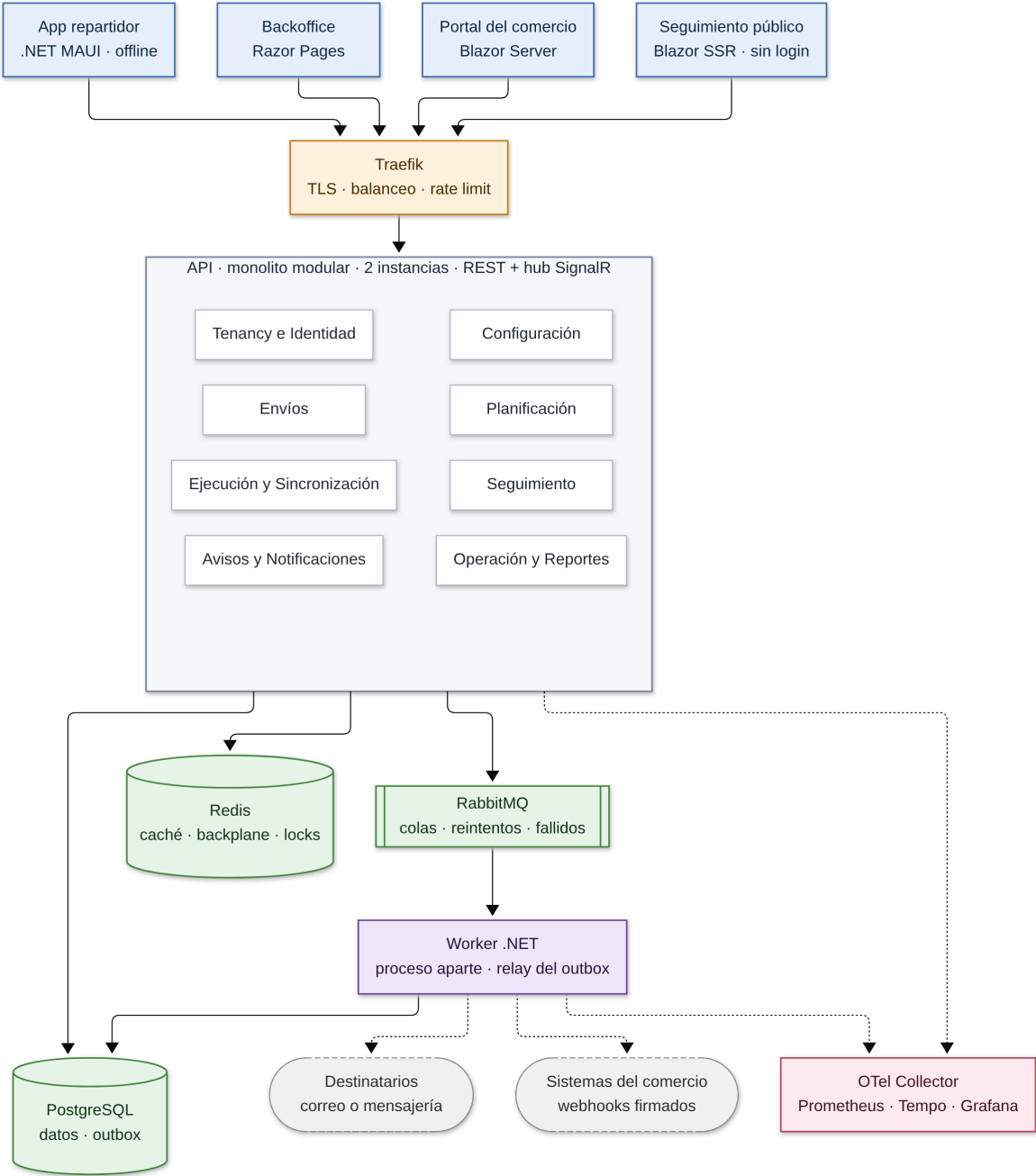


Figura 1: Arquitectura lógica. Cuatro clientes entran por Traefik a la API modular; el Worker es un proceso aparte que solo consume de la cola. Los sistemas externos van en línea punteada.

`ITenantContext`, publicador de eventos. Infrastructure las implementa con EF Core, Redis y RabbitMQ. La API y el Worker solo registran dependencias e invocan Handlers.

Una prueba con `NetArchTest` en el pipeline falla si `Domain` o `Application` referencian `Microsoft.EntityFrameworkCore`, `Microsoft.AspNetCore` o `Infrastructure`, y si un módulo usa clases internas de otro.

1.3 Aplicaciones desplegables

- **API** (ASP.NET Core): endpoints REST, hub de SignalR, único punto de autorización. Hospeda todos los módulos.
- **Backoffice** (Razor Pages), **Portal del comercio** (Blazor `InteractiveServer`) y **Seguimiento público** (Blazor SSR estático): consumen la API con clientes HTTP tipados; no acceden a la base.
- **Worker** (.NET Worker Service): consume la cola y ejecuta procesamiento de eventos móviles, notificaciones, webhooks, indicadores y cierre diario. Reutiliza los proyectos `Application` e `Infrastructure`, pero se despliega por separado.
- **App del repartidor** (.NET MAUI, MVVM): SQLite cifrado local, sincroniza contra la API.

1.4 Comunicación entre módulos

Dentro del proceso es síncrona mediante las interfaces públicas de cada módulo. Todo efecto hacia afuera (notificar, avisar al comercio, recalcular indicadores) pasa por eventos de integración escritos en la tabla `outbox` en la misma transacción que el cambio, y publicados a RabbitMQ.

2. Arquitectura de despliegue previsto

Todo corre en contenedores Docker orquestados con `docker compose`, idéntico en local y en la nube. En la nube se despliega sobre una VM Linux de Azure for Students aprovisionada con OpenTofu.

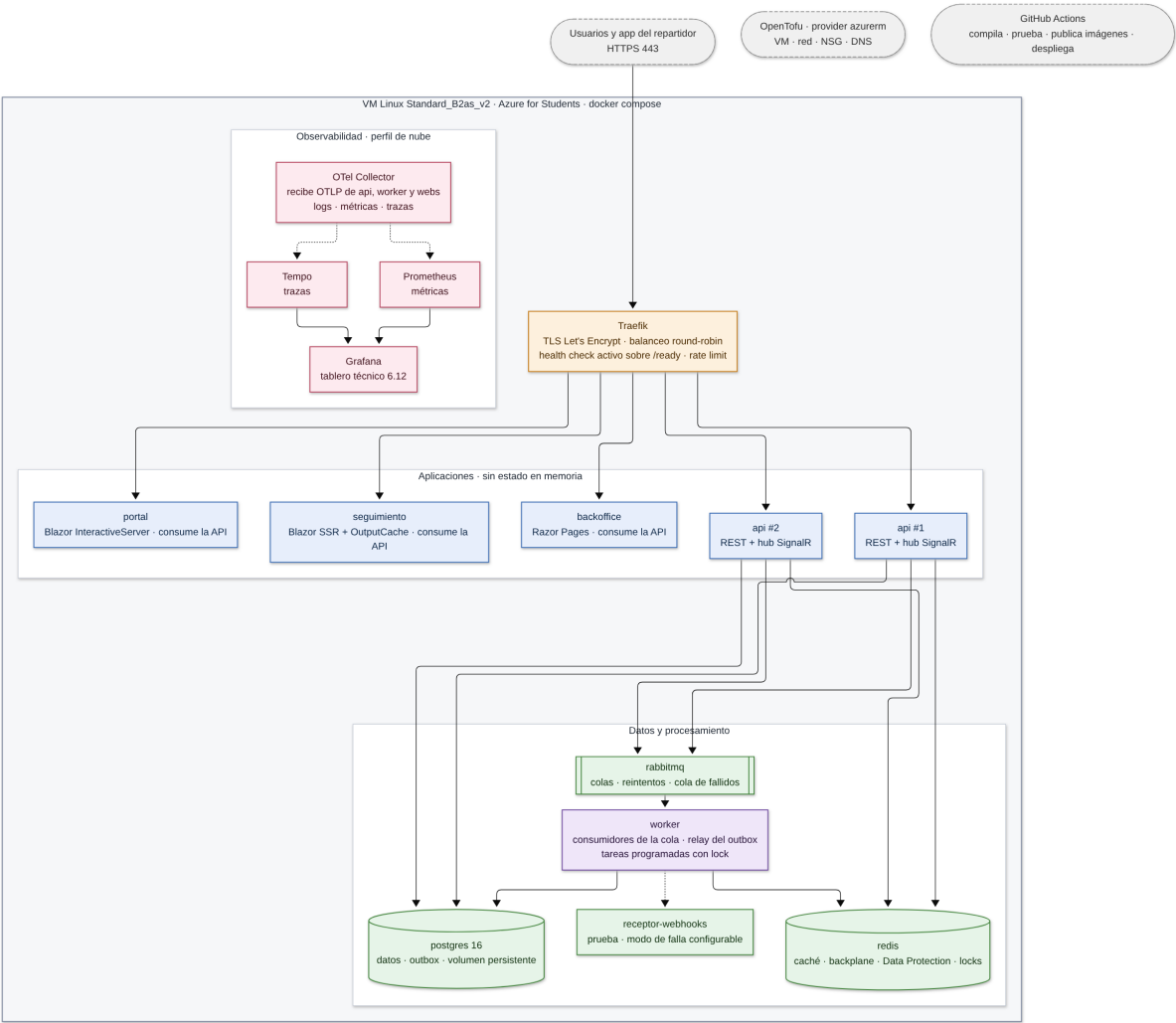


Figura 2: Despliegue previsto. Una VM con todo el `compose`; Traefik balancea las dos instancias de la API. OpenTofu y GitHub Actions son herramientas externas a la VM.

2.1 Contenedores y réplicas

Contenedor	Rép.	Función
traefik	1	Proxy inverso, TLS con Let's Encrypt, balanceo round-robin entre las instancias de la API, health checks activos, rate limit y cabeceras seguras
api	2	API REST + hub SignalR, sin estado en memoria; <code>/health</code> , <code>/ready</code> , <code>/live</code>
backoffice	1	Razor Pages del operador
portal	1	Blazor <code>InteractiveServer</code> del comercio
seguimiento	1	Blazor SSR estático con <code>OutputCache</code> , público

Contenedor	Rép.	Función
worker	1	Consumidores de la cola, relay del outbox, tareas programadas con lock distribuido
postgres	1	Base transaccional (volumen persistente, backups)
redis	1	Caché distribuido, backplane de SignalR, llaves de Data Protection, locks
rabbitmq	1	Colas de trabajo, reintentos y cola de fallidos
otel-collector, prometheus, tempo, grafana	1 c/u	Observabilidad en el perfil de nube
receptor-webhooks	1	Receptor de prueba de avisos a comercios, con modo de falla configurable

En desarrollo, el perfil de **docker compose** reemplaza Prometheus, Tempo y Grafana por el **Aspire dashboard**, que recibe el mismo OTLP del Collector. La aplicación no cambia: el backend de visualización es una decisión de perfil, no de código.

2.2 Continuidad ante la caída de una instancia

Traefik saca del pool a la instancia que falla su chequeo **/ready**. Como la API no guarda estado en memoria —sesión, caché y llaves de Data Protection viven en Redis—, la otra instancia atiende sin pérdida. SignalR usa el backplane de Redis, así que un mensaje emitido desde una instancia llega a los clientes conectados a la otra.

2.3 Nube

Azure for Students: una VM Linux **Standard_B2as_v2** alcanza para el compose completo, con crédito de estudiante, sin tarjeta de crédito y con corte automático al agotarse. La VM, la red, las reglas del grupo de seguridad y el DNS se definen en OpenTofu con el provider **azurerm** y variables por ambiente; el ambiente se destruye y se reconstruye con un comando.

3. Modelo de dominio preliminar

Entidades clave y relaciones, agrupadas en cinco áreas. Es un modelo **preliminar**: fija las entidades y sus relaciones, no el esquema físico. Las claves de inquilino y los detalles de persistencia se resuelven en la implementación, según el ADR-002.

3.1 Áreas y entidades

Área	Entidades	Qué resuelve
1. Organización	Operador, Comercio, CuentaComercial, Usuario, Rol	Quién es inquilino, quién es su cliente y quién opera el sistema
2. Configuración	ZonaCobertura, FranjaHoraria, CuadroTarifario, VersionTarifaria, ConfiguracionOperativa, VersionReglas	Lo que cada operador parametriza sin tocar código
3. Envíos	Envio, Bulto, Destinatario, Direccion, EventoEnvio, IntentoEntrega, PruebaEntrega, MotivoNoEntrega, Incidencia, Devolucion	El ciclo de vida del envío y su evidencia
4. Distribución	Ruta, Parada, AsignacionRuta, Repartidor, Vehiculo	Cómo se planifica y ejecuta el reparto
5. Comunicación y facturación	SuscripcionAviso, Notificacion, Liquidacion	Avisos al destinatario y al comercio, y el cierre económico

3.2 Relaciones que sostienen el diseño

CuentaComercial es el corazón del multitenancy. Un Comercio es una identidad global que puede trabajar con varios operadores; cada vínculo operador–comercio es una **CuentaComercial**. Por eso el **Envio** cuelga de la **CuentaComercial** y no del **Comercio**: así un mismo comercio opera con dos operadores sin que ninguno vea los envíos del otro. Es la contraparte en el modelo de la decisión del ADR-002.

La configuración se versiona, no se pisa. **CuadroTarifario** versiona en **VersionTarifaria** y **ConfiguracionOperativa** versiona en **VersionReglas**, ambas con **vigenciaDesde** y **vigenciaHasta**. El **Envio** queda atado a la versión vigente al momento de su alta, de modo que cambiar una tarifa o subir el máximo de intentos no reescribe la historia de los envíos ya tomados. De ahí salen las condiciones de la máquina de estados que hablan de “la versión de reglas del envío”.

El EventoEnvio es inmutable y es la fuente del seguimiento. Cada transición válida de la máquina de estados agrega un **EventoEnvio**; de ahí se alimentan el seguimiento público y, por **SuscripcionAviso**, las **Notificacion** al destinatario y los avisos a los comercios.

AsignacionRuta es la relación entre Envio y Ruta, y no un vínculo directo: un envío puede ser asignado, quitado y reasignado varias veces, y el atributo **activa** distingue la asignación vigente de las históricas. Es lo que permite las transiciones **AsignarARuta** y **QuitarDeRuta** sin perder el historial.

IntentoEntrega separa el intento de su resultado. Cada intento referencia a lo sumo una **PruebaEntrega** (si entregó) o un **MotivoNoEntrega** del catálogo del operador (si no). Contar los intentos de un envío es lo que decide si **RetornarADeposito** o **IniciarDevolucion**.

3.3 Dónde entra el aislamiento entre inquilinos

El modelo no dibuja las claves de inquilino para no repetir **OperadorId** en treinta cajas, pero según el ADR-002 toda entidad que pertenece a un operador la lleva, y las que pertenecen a un

comercio llevan además **ComercioId**. En el diagrama eso se lee por las asociaciones: todo cuelga, directa o transitivamente, de **Operador** o de **CuentaComercial**.

3.4 Pendientes del modelo preliminar

Se resuelven al implementar; se listan para que no se pierdan:

- **Token de seguimiento público.** El RF 25 consulta por un token opaco; falta el atributo en **Envio** o una entidad **EnlaceSeguimiento** con su índice.
- **Contador de intentos en Envio.** Las condiciones de la máquina de estados comparan “intentos realizados” contra el máximo de la versión de reglas. Hoy es derivable contando **IntentoEntrega**, pero conviene materializarlo.
- **Fecha y franja de reprogramación.** El estado Reprogramado supone una nueva fecha acordada que todavía no tiene atributo.
- **Repartidor y Usuario.** Hoy son clases separadas; el repartidor se autentica con su propio token, así que falta decidir si especializa a **Usuario** o lo referencia.

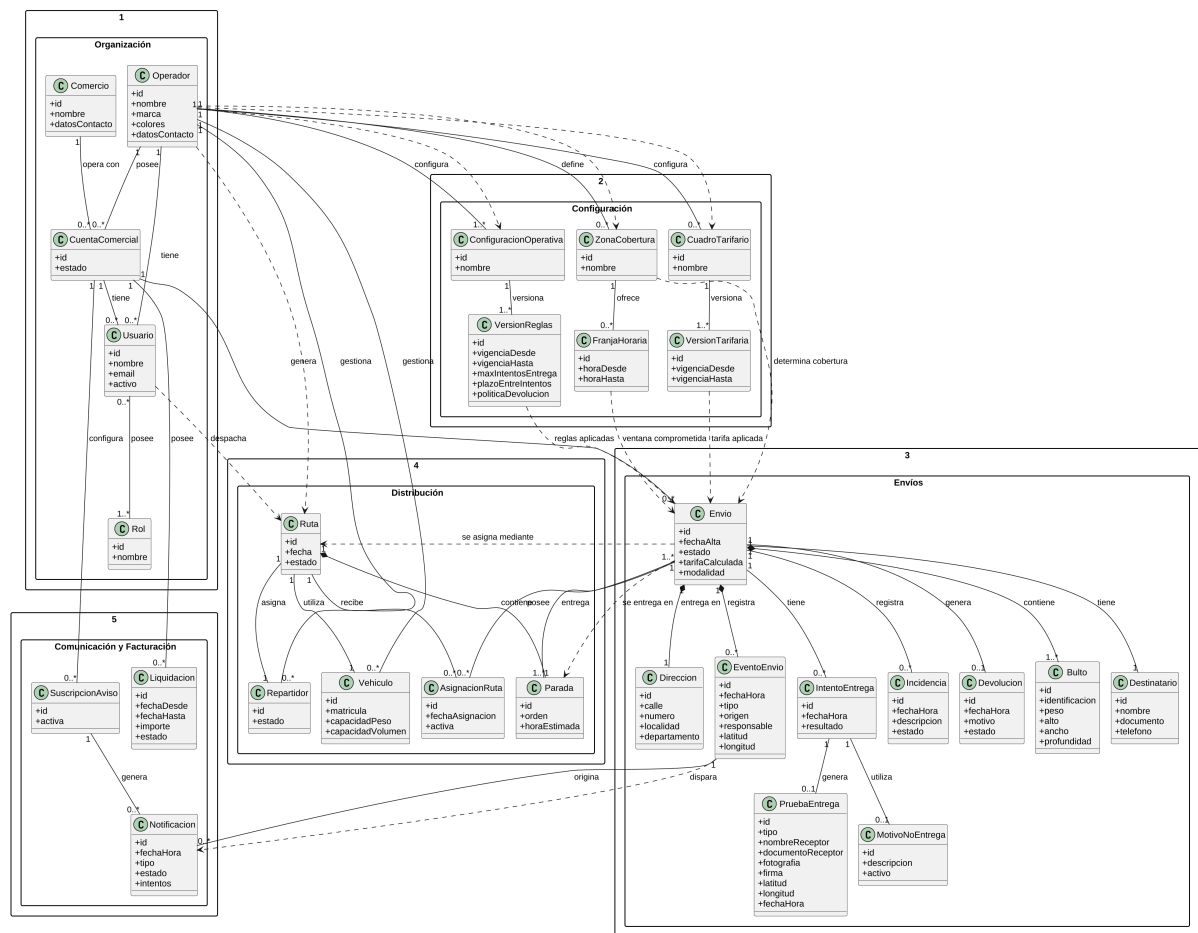


Figura 3: Modelo de dominio preliminar. Veintinueve entidades en cinco áreas; las flechas punteadas cruzan de un área a otra.

4. Máquina de estados del envío

El estado del envío solo cambia mediante `Envio.Aplicar(disparador)`, que consulta una tabla declarativa de transiciones; cualquier otra combinación lanza una excepción de dominio. Cada transición válida genera un `EventoEnvio` y un evento de integración en el outbox.

Leyenda

Color	Significado
Azul	Estado operativo del recorrido normal
Violeta	Estado del circuito de devolución
Ámbar	Estado de excepción, alcanzable desde cualquier estado no final
Verde, borde grueso	Estado final

4.1 Transiciones válidas

Desde	Disparador	Hacia	Actor	Condición
Admitido	RecibirEnDeposito	En depósito	Operario de depósito	Escaneo de todos los bultos; las discrepancias con lo declarado quedan registradas
En depósito	AsignarARuta	Asignado a ruta	Despachador	Primer intento (intentos realizados = 0); ruta válida en paradas, peso, volumen y franja; sin otra asignación activa
Asignado a ruta	QuitarDeRuta	En depósito	Despachador	La ruta todavía no fue despachada
Asignado a ruta	Despachar	En tránsito	Repartidor	Bulto escaneado al cargar el vehículo
En tránsito	Entregar	Entregado	Repartidor	Prueba de entrega completa según la versión de reglas del envío
En tránsito	RegistrarIntentoFallido	No entregado	Repartidor	Motivo del catálogo del operador + evidencia; suma 1 a intentos realizados
No entregado	RetornarADeposito	En depósito	Operario de depósito	Intentos realizados < máximo de la versión de reglas y el motivo no exige devolución; se escanea en la rendición
No entregado	IniciarDevolucion	En devolución	Sistema	Intentos realizados = máximo, o el motivo exige devolución (por ejemplo, dirección inexistente)

Desde	Disparador	Hacia	Actor	Condición
En depósito	Reprogramar	Reprogramado	Sistema, despachador o destinatario	Hubo al menos un intento fallido; nueva fecha y franja acordadas, respetando el plazo mínimo entre intentos
Reprogramado	AsignarARuta	Asignado a ruta	Despachador	La fecha de la ruta coincide con la fecha reprogramada
En devolución	ConfirmarDevolucionDevuelto		Operario de depósito	El comercio recibe el bulto y firma
En depósito, Asignado a ruta, En tránsito, No entregado, Reprogramado, En devolución	DeclararExtraviado	Extraviado	Despachador	Justificación obligatoria
Extraviado	Reaparecer	En depósito	Operario de depósito	El bulto se escanea en depósito

4.2 Notas de diseño

Entregado y Devuelto son estados finales. Tras un intento fallido, el paquete vuelve físicamente al depósito antes de reprogramarse; por eso la asignación directa desde En depósito vale solo para el primer intento, y los siguientes deben pasar por Reprogramado, que representa una nueva fecha ya acordada. El envío guarda la cantidad de intentos realizados para evaluar estas condiciones. Si el destinatario pide reprogramar antes del primer intento, cambia la franja comprometida pero no el estado.

Implementación prevista: la librería Stateless, que permite exportar la configuración a un gráfico y escribir una prueba unitaria por transición válida y por transición inválida.

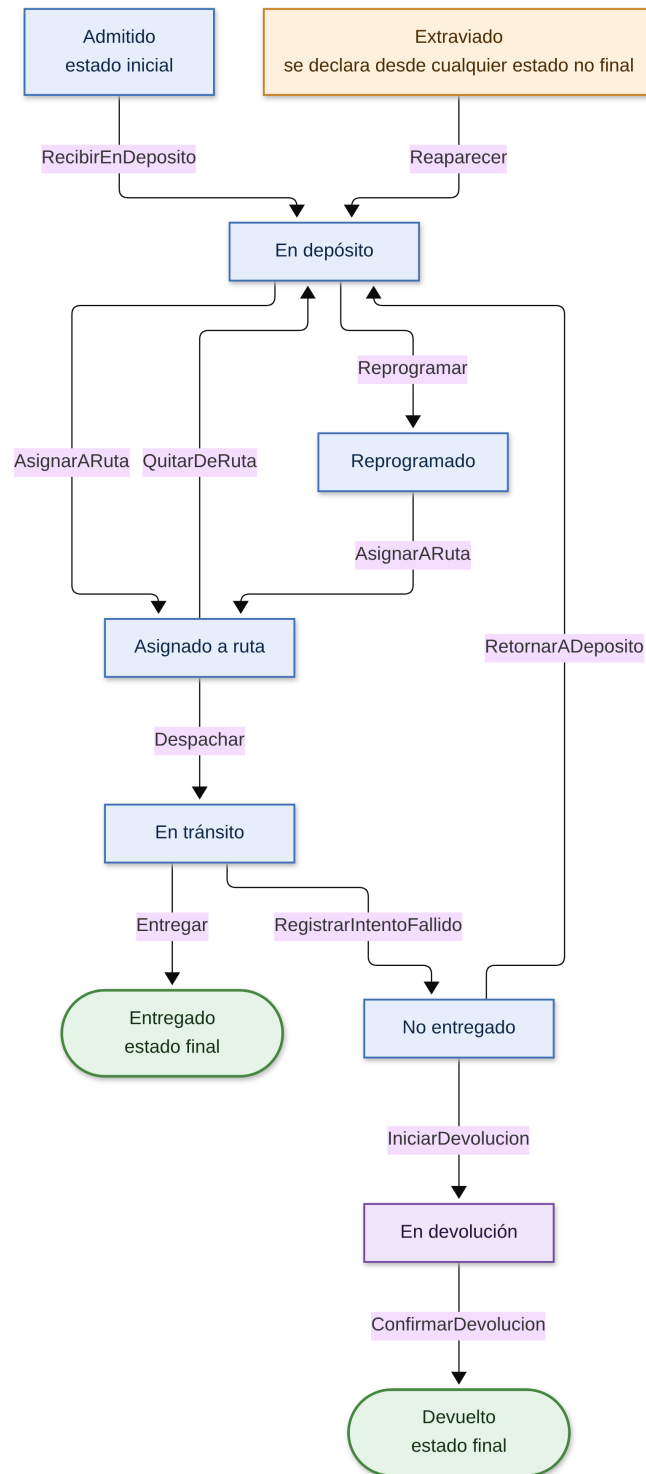


Figura 4: Máquina de estados del envío: diez estados, dos finales. La transición **DeclararExtraviado** no se dibuja como doce flechas; el nodo Extraviado declara en su texto que se alcanza desde cualquier estado no final.

5. Estrategia de multitenancy

Aislamiento por fila en una base compartida: toda entidad que pertenece a un operador lleva `OperadorId`, y las que pertenecen a un comercio llevan además `ComercioId`. La justificación completa y las alternativas descartadas están en el ADR-002 (sección 9).

5.1 Resolución del inquilino en tiempo de ejecución

Canal	Cómo se resuelve
Backoffice, Portal, API autenticada	Claim <code>operador_id</code> del token; para usuarios de comercio, también <code>comercio_id</code>
App del repartidor	Claims del token del repartidor, validados en cada sincronización
Seguimiento público	El token opaco del enlace se busca en un índice y determina operador y envío
Worker	El <code>OperadorId</code> viaja dentro de cada mensaje y se fija en el contexto antes de procesarlo
SignalR	Al conectar, el usuario se une al grupo <code>operador:{id}</code> ; nunca recibe mensajes de otro grupo

5.2 Aislamiento en lecturas y escrituras

Un servicio `ITenantContext` con alcance por solicitud alimenta filtros globales de EF Core (`HasQueryFilter`) por `OperadorId` y, cuando el usuario es de un comercio, por `ComercioId`. El uso de `IgnoreQueryFilters` se restringe a un único repositorio de administración de plataforma, verificado por prueba de arquitectura.

En escrituras, un interceptor de `SaveChanges` asigna `OperadorId` a toda entidad nueva y rechaza cualquier modificación cuyo `OperadorId` no coincida con el contexto actual.

Defensa en profundidad. Row-Level Security de PostgreSQL con una variable de sesión, para que un error en el código no alcance a filtrar datos.

5.3 Pruebas automatizadas de aislamiento

Con Testcontainers se crean dos operadores con datos; para cada endpoint de lectura se verifica que el operador A no obtiene datos de B, que un comercio no ve envíos de otro comercio del mismo operador, y que una escritura con Id ajeno devuelve 404.

5.4 Diferenciación por inquilino

Marca (logo, colores, contacto), tarifario y reglas se leen de la configuración del operador, con caché en Redis e invalidación al publicar una nueva versión. Las migraciones son únicas para todos; los datos iniciales crean dos operadores de demostración con configuraciones distintas.

6. Stack tecnológico y entorno de desarrollo

Todo el stack es .NET 10 sobre contenedores, con componentes open source que caben en el crédito gratuito de la nube. No quedan opciones abiertas: las que en la primera versión del análisis figuraban “a confirmar” están cerradas acá.

6.1 Stack confirmado

Área	Elección	Motivo breve
Plataforma	.NET 10	Exigido por la letra
Base transaccional	PostgreSQL 16 + EF Core 10 (Npgsql)	Mismo motor que la demo Support; índices parciales, <code>xmin</code> como token de concurrencia y Row-Level Security disponible
Validación	FluentValidation	Igual que la demo: un Validator por Command o Query
Caché y coordinación	Redis	Con dos instancias de API el <code>IMemoryCache</code> no se comparte; Redis sí. También backplane de SignalR, llaves de Data Protection y locks distribuidos
Mensajería	RabbitMQ	MOM con broker visto en clase; colas de trabajo con reintentos y cola de fallidos
Mensajería y outbox	MassTransit v8	Outbox transaccional sobre EF Core, idempotencia por inbox, reintentos con espera creciente y cola de fallidos ya resueltos. Versión fijada en <code>Directory.Packages.props</code> : v8 es Apache-2.0, v9 cambia de licencia
Máquina de estados	Stateless	Transiciones declarativas, verificables y exportables a gráfico
Identidad	ASP.NET Core Identity	La letra lo admite como almacén de usuarios; cookies en las webs, tokens bearer en la API y la app; claims de operador y comercio
API	Minimal API con OpenAPI nativo + Scalar	Mismo stack que la demo de patrones
Backoffice del operador	ASP.NET Core Razor Pages	Exigido por la letra
Portal del comercio	Blazor InteractiveServer	Interactividad con estado en el servidor; funciona tras el balanceador con las llaves de Data Protection y el backplane de SignalR en Redis
Seguimiento público	Blazor SSR estático + OutputCache	Es la superficie de mayor tráfico y la única anónima: render estático evita un circuito y un WebSocket por visitante
Tiempo real	SignalR + backplane Redis	Un mensaje emitido desde una instancia llega a los clientes conectados a la otra
Tareas programadas	Alojadas solo en el Worker (1 réplica) + lock en Redis	La duplicación que advierte 6.13 solo aparece si se alojan en la API escalada; el lock cubre el caso de reinicio solapado

Área	Elección	Motivo breve
Móvil	.NET MAUI con CommunityToolkit.Mvvm, ZXing.Net.Maui, Microsoft.Data.Sqlite con SQLitePCLRaw.bundle_e_sqlcipher, Plugin.Fingerprint, Firebase Cloud Messaging	Mismas librerías que la demo BeatScan, cambiando el bundle de SQLite por uno con cifrado
Instrumentación	Serilog + OpenTelemetry → OTel Collector	Exigido por 6.12; el Collector desacopla la aplicación del backend de visualización
Visualización — desarrollo	Aspire dashboard	Logs, métricas y trazas correlacionadas en un contenedor y sin configuración
Visualización — nube	Prometheus + Tempo + Grafana	Sostiene el tablero técnico con paneles propios e historial, que el Aspire dashboard no da
Proxy inverso y TLS	Traefik + Let's Encrypt	Único proxy del sistema: balanceo con health checks activos, certificados automáticos, rate limit y cabeceras seguras
Pruebas	xUnit, Testcontainers, NetArchTest	Unitarias de dominio, integración contra Postgres y RabbitMQ reales, y la prueba de arquitectura que exige 6.1
Infraestructura como código	OpenTofu + provider azurerm	Compatible con Terraform, sin licencia restrictiva; mantiene posible un cambio de proveedor
CI/CD	GitHub Actions	Compila, prueba, construye imágenes y despliega
Nube	Azure for Students — VM Linux Standard_B2as_v2	IaaS con crédito de estudiante, sin tarjeta de crédito y con corte automático al agotarse

6.2 Opcionales elegidos (12 puntos)

Opcional	Tecnología	Motivo breve
API pública para comercios (3)	Asp.Versioning + OpenAPI/Scalar, claves por comercio, rate limit diferenciado	Extiende el alta por API del RF 7 sobre el mismo Minimal API
Notificaciones remotas en producción (3)	Firebase Cloud Messaging	Ya es el canal push de la app del repartidor (6.11)
Pruebas de carga (2)	k6 en contenedor	Escenarios como código, se ejecuta desde el pipeline contra el ambiente remoto
Pruebas de resiliencia (2)	docker compose stop de la dependencia + Polly para el degradado; Toxiproxy si hace falta latencia o pérdida	Interrumpe caché, cola, base y receptor externo sin tocar la aplicación

Cobertura de pruebas (2)	coverlet + ReportGenerator, reporte publicado por GitHub Actions	Cobertura verificable sobre dominio y aplicación
--------------------------	---	---

Playwright queda fuera del stack: las pruebas de extremo a extremo no están entre los opcionales elegidos.

6.3 Traefik como único proxy inverso

Se usa un solo proxy en el borde, Traefik, y no hay contenedor nginx.

- **Descubrimiento por labels de Docker.** api escala a dos réplicas sin editar ningún upstream. Con nginx el bloque `upstream` se escribe y se recarga a mano cada vez que cambia la cantidad de instancias.
- **Health checks activos sobre /ready.** Traefik saca del pool a la instancia que falla el chequeo *antes* de enrutarle tráfico. nginx open source solo tiene chequeos **pasivos** (`max_fails`, `proxy_next_upstream`): reacciona después de haber fallado peticiones reales. Esto pesa directamente en el primer escenario anunciado para la defensa: interrumpir una instancia de la API con tráfico en curso y verificar la continuidad del servicio.
- **ACME integrado.** Certificado válido de Let's Encrypt sin contenedor `certbot`, sin cron de renovación y sin volumen compartido de certificados.
- **Rate limit y cabeceras seguras declarativos** como middlewares, que es parte de lo que exige 6.3.
- **Un contenedor en lugar de dos**, en una VM que ya corre el resto del compose.

Alternativas descartadas: **nginx como único proxy** (familiar y de configuración explícita, pero obliga a un sidecar `certbot` con su renovación, a mantener los upstreams a mano y a conformarse con health checks pasivos); **nginx delante de Traefik** (dos proxies que configurar, explicar y depurar; solo se justificaría por una caché HTTP para el seguimiento público, que ya se resuelve con `OutputCache` sobre Redis dentro de la propia aplicación); y **YARP** (cumple la misma función en .NET, pero no hace ACME: haría falta otro componente para los certificados).

6.4 Observabilidad: dos backends, una sola instrumentación

La aplicación habla **únicamente OTLP contra el OpenTelemetry Collector**. El backend de visualización es, por lo tanto, una decisión de perfil de `docker compose` y no de código: el mismo binario instrumentado sirve a los dos.

- **En desarrollo**, el Aspire dashboard recibe ese OTLP y muestra logs, métricas y trazas correlacionadas en un contenedor, sin configuración ni dashboards que provisionar.
- **En la nube**, Prometheus, Tempo y Grafana sostienen el tablero técnico que exige 6.12 (tiempo medio de respuesta, percentiles 95 y 99, tasa de errores, profundidad de la cola, sincronizaciones pendientes, avisos a comercios fallidos y relación de aciertos del caché). El Aspire dashboard no puede cubrirlo: no admite paneles propios, no persiste y se vacía al reiniciar.

El identificador de correlación viaja por el contexto de OpenTelemetry desde la app del repartidor hasta el Worker, con los mensajes de MassTransit propagando el contexto de traza.

6.5 Entorno de desarrollo

- JetBrains Rider 2025.3+ o Visual Studio 2026, como en las demos de clase.

- Las mismas convenciones de la demo Support: `global.json` fija el SDK de .NET 10, `Directory.Packages.props` centraliza las versiones de NuGet y `.config/dotnet-tools.json` fija `dotnet-ef` como herramienta local.
- También de la demo: `AuditableEntity` con un interceptor de `SaveChanges` para auditoría, `DbSeeder` idempotente con los dos operadores de demostración, y health checks.
- Docker Desktop; `docker compose up` levanta todo el entorno con datos de demostración, incluido el Aspire dashboard como backend de observabilidad local.
- Repositorio único en GitHub, rama `main` protegida, pull requests con revisión de al menos un integrante y pipeline en verde.
- Secretos en `dotnet user-secrets` en local, GitHub Secrets en CI y variables de entorno (`.env` fuera del repositorio) en el servidor.
- Android: emulador o dispositivo físico para MAUI, con los workloads indicados en la demo BeatScan; el seguimiento en segundo plano se prueba en dispositivo físico.

6.6 Trazabilidad contra los requerimientos no funcionales

Autocontrol del stack: cada requerimiento obligatorio de la sección 6 de la letra con los componentes que lo cubren. Un requerimiento sin fila sería un agujero en el stack.

RNF	Componentes del stack
6.1 Plataforma y arquitectura	.NET 10; Docker y <code>docker compose</code> para todo el entorno, servicios de soporte incluidos; NetArchTest para la prueba de arquitectura
6.2 Capa de presentación web	Razor Pages (Backoffice); Blazor <code>InteractiveServer</code> (Portal); Blazor SSR estático + <code>OutputCache</code> (Seguimiento público)
6.3 Seguridad y autenticación	ASP.NET Core Identity; TLS, rate limit y cabeceras seguras en Traefik; CORS de ASP.NET Core; SQLCipher en el dispositivo; <code>dotnet user-secrets</code> , GitHub Secrets y <code>.env</code> fuera del repositorio
6.4 Persistencia y datos	PostgreSQL 16 + EF Core 10 con migraciones vía <code>dotnet-ef</code> ; <code>xmin</code> de Npgsql como token de concurrencia optimista; SQLite cifrado en el móvil
6.5 Multitenancy	Filtros globales de EF Core alimentados por <code>ITenantContext</code> + interceptor de <code>SaveChanges</code> ; Testcontainers para las pruebas de aislamiento; Row-Level Security de PostgreSQL como defensa adicional
6.6 Configurabilidad y parametrización	Versiones de tarifario y de reglas en EF Core, con caché e invalidación en Redis
6.7 Caché distribuido	Redis vía <code>IDistributedCache</code> y <code>OutputCache</code> ; métricas de aciertos y fallos por OpenTelemetry hacia Grafana
6.8 Mensajería y segundo plano	RabbitMQ + MassTransit v8: outbox sobre EF Core, inbox para idempotencia, reintentos con espera creciente y cola de fallidos; Worker Service independiente; consulta y reintento de fallidos desde el Backoffice
6.9 Avisos automáticos a los comercios	<code>HttpClient</code> con Polly, firma HMAC del mensaje, reintentos y cola de fallidos de MassTransit; receptor de prueba propio con modo de falla configurable
6.10 Comunicación en tiempo real	SignalR con backplane de Redis, con grupos por operador

RNF	Componentes del stack
6.11 Aplicación móvil	.NET MAUI con MVVM (CommunityToolkit.Mvvm), ZXing.Net.Maui, SQLite con SQLCipher, Plugin.Fingerprint, Firebase Cloud Messaging
6.12 Observabilidad	Serilog + OpenTelemetry → Collector; Aspire dashboard en desarrollo; Prometheus, Tempo y Grafana en la nube
6.13 Escalabilidad y balanceo	Traefik balanceando dos instancias de API con health checks activos; /health, /ready y /live de ASP.NET Core; apagado controlado; Data Protection en Redis; tareas programadas alojadas solo en el Worker
6.14 Despliegue e IaC	Azure for Students sobre VM Standard_B2as_v2 ; OpenTofu con provider azurerm ; Traefik + Let's Encrypt para HTTPS válido; GitHub Actions para compilar, probar y desplegar
6.15 Calidad	xUnit (dominio), Testcontainers (integración y aislamiento entre inquilinos), NetArchTest (arquitectura), coverlet + ReportGenerator, todo ejecutado por GitHub Actions

7. Plan de trabajo

Cada integrante es responsable de un área y de sus ADR; responder por un área no impide programar en otras, pero el responsable la debe poder explicar y defender. El reparto es una propuesta inicial que el equipo puede ajustar según afinidades.

7.1 Asignación de responsabilidades

Integrante	Área de responsabilidad	ADR a cargo	Opcionales
Juliana	Dominio y reglas: Envíos, máquina de estados, configuración versionada (tarifas y reglas), Planificación, reportes y liquidación, prueba de arquitectura	ADR-001 Estilo arquitectónico	API pública para comercios (3), Cobertura de pruebas (2)
Martina	Multitenancy, identidad y seguridad; Backoffice, Portal y Seguimiento; caché distribuido; tablero en vivo con SignalR	ADR-002 Multitenancy; modo de renderizado de Blazor; caché e invalidación	
Matías	Mensajería: outbox, Worker, reintentos y cola de fallidos; avisos a comercios y receptor de prueba; notificaciones; observabilidad; infraestructura como código, CI/CD y balanceo	Consistencia entre persistencia y eventos	Pruebas de carga (2), Pruebas de resiliencia (2)
Cecilia	App MAUI: MVVM, SQLite cifrado, escaneo, pruebas de entrega, posición en segundo plano, biometría; sincronización diferida y conflictos	Resolución de conflictos de sincronización	Notificaciones remotas en producción (3)

Opcionales elegidos: 12 puntos. Se eligieron por sinergia con requerimientos obligatorios: las notificaciones push ya se necesitan para reasignaciones (6.11), la API pública extiende el alta por API (RF 7) y las pruebas de carga y resiliencia validan el balanceo y los reintentos (6.8, 6.13).

7.2 Cronograma por monitoreo

Fecha	Objetivo	Juliana	Martina	Matías	Cecilia
8/10	Esqueleto de extremo a extremo	Solución, módulos y alta de envío	Login básico, Portal da de alta, Backoffice lista	docker compose completo, pipeline CI	Proyecto MAUI que se autentica
15/10	Estados, multitenancy, autorización	Máquina de estados con pruebas	Filtros por inquilino, dos operadores, perfiles	Outbox escribiendo eventos	SQLite cifrado y modelo local
22/10	Móvil sin conexión, caché	Hojas de ruta y validación	Caché con métrica de aciertos	Métricas en Grafana	Descarga de ruta, entregas sin conexión, sincronización

Fecha	Objetivo	Juliana	Martina	Matías	Cecilia
29/10	Worker, avisos, tiempo real + cambio de requerimientos	Reglas versionadas en caliente	Tablero en vivo	Worker, webhooks con reintentos y fallidos	Conflictos y push
5/11	Nube con IaC, 2 instancias, observabilidad	Reportes	Seguimiento público protegido	Despliegue con OpenTofu y balanceo	Posición en segundo plano y biometría
12/11	Opcionales y ensayo	API pública, cobertura	Pulido de UI y pruebas de aislamiento	Carga y resiliencia	Push en producción

7.3 Prácticas del equipo

Reunión corta dos veces por semana y revisión de PR cruzada entre áreas, para que todos conozcan el código de los demás antes de la defensa. Desde el primer día se lleva el registro de horas por integrante y actividad, y la bitácora de uso de IA por componente.

8. ADR-001 — Estilo arquitectónico y organización del código

Responsable: Juliana • **Estado:** aceptada • **Fecha:** 4/10/2026

Contexto y restricciones

La letra impone un monolito modular para la API y las webs, y un Worker independiente que se comunica solo por la cola. Exige que el dominio no dependa de infraestructura ni frameworks, verificado por una prueba automática. El 29/10 llega un cambio de requerimientos no anunciado, y cada integrante debe poder explicar su parte. El equipo tiene cuatro personas y unas 430 horas en total.

Decisión

Clean Architecture con organización Vertical Slice, el mismo esquema de la demo Support vista en clase, extendido a un monolito modular. Hay un proyecto por capa (Domain, Application, Infrastructure) compartido por todos los módulos. Dentro de Domain y Application cada módulo de negocio (Envíos, Planificación, Configuración, etc.) es una carpeta. Dentro de Application cada caso de uso es un slice con su Command o Query, su Handler y su Validator de FluentValidation. Application accede a los datos solo mediante `IUnitOfWork` y repositorios definidos como interfaces, que Infrastructure implementa con EF Core. La API y el Worker invocan Handlers; las webs consumen la API. La regla se verifica con NetArchTest en el pipeline.

```
src/
  UltimaMilla.Domain/
    Common/           AuditableEntity, ITenantEntity
    Envios/            Envio, Bulto, EstadoEnvio, MaquinaEstadosEnvio
    Planificacion/     Ruta, Parada, Vehiculo
    Configuracion/     CuadroTarifario, VersionTarifaria,
                      ConfiguracionOperativa, VersionReglas, ZonaCobertura
  UltimaMilla.Application/
    Abstractions/      IUnitOfWork, IEnvioRepository, ITenantContext
    Common/            PagedResult, QueryParameters
    Envios/
      Commands/CrearEnvio/          CrearEnvioCommand, Handler, Validator
      Commands/RegistrarIntentoFallido/ ...
      Queries/ListarEnvios/         ListarEnviosQuery, Handler
    Planificacion/
      Commands/AsignarARuta/        ...
  UltimaMilla.Infrastructure/
    Persistence/       AppDbContext, Configurations, Migrations, Seed, Interceptors
    Repositories/      EnvioRepository, RutaRepository, UnitOfWork
    Messaging/         Outbox, publicador RabbitMQ
  UltimaMilla.Api/     Minimal API + hub SignalR
  UltimaMilla.Web.Backoffice/  Razor Pages
  UltimaMilla.Web.Portal/     Blazor
  UltimaMilla.Web.Seguimiento/ Blazor
  UltimaMilla.Worker/        Worker Service
  UltimaMilla.Mobile/       .NET MAUI (MVVM)
tests/
  Architecture.Tests, Domain.Tests, Integration.Tests, Tenancy.Tests
```

Alternativas consideradas

- Un juego de proyectos Domain, Application e Infrastructure por cada módulo. Da límites más fuertes, porque el compilador impide que un módulo use el interior de otro. Pero

serían más de 24 proyectos, excesivo para cuatro personas y nueve semanas; la separación por carpetas más la prueba de arquitectura por namespace alcanza.

- **Capas organizadas por tipo técnico, sin Vertical Slice** (todos los handlers juntos, todos los validadores juntos). Descartada: un cambio en un caso de uso obliga a tocar archivos dispersos, y eso complica absorber el cambio de requerimientos del 29/10.
- **Las webs invocan los Handlers directamente, como las Razor Pages de la demo.** Descartada: acá hay cuatro clientes (Backoffice, Portal, Seguimiento y la app móvil) más la API para comercios. Si todos pasan por la API, la autorización y el aislamiento entre operadores se controlan en un único lugar.
- **Microservicios.** Descartada: la letra pide monolito modular, y multiplicaría despliegues, bases y fallas de red para un equipo de cuatro.

Consecuencias

- A favor: es el mismo esquema de la demo de clase, así que todo el equipo parte de un modelo conocido. Agregar un caso de uso es agregar una carpeta (un slice) sin tocar los demás, lo que debería contener el cambio del 29/10 en pocos slices.
- En contra: los límites entre módulos dependen de la disciplina del equipo y de la prueba de arquitectura, no del compilador. Cada caso de uso lleva más archivos (Command, Handler, Validator, DTO).
- Que las webs pasen por la API agrega una llamada HTTP respecto de la demo; a cambio, hay una única puerta de autorización.
- Lo que dificulta a futuro: separar un módulo como servicio independiente (por ejemplo, el opcional del Seguimiento) exige mover su carpeta a proyectos propios.

9. ADR-002 — Estrategia de multitenancy

Responsable: Martina • **Estado:** aceptada • **Fecha:** 4/10/2026

Contexto y restricciones

Un único despliegue atiende a varios operadores logísticos, que no deben ver datos entre sí. Hay un segundo nivel: dentro de cada operador, un comercio solo ve lo suyo, y un mismo comercio puede trabajar con varios operadores. Se exige resolución del inquilino en tiempo de ejecución, pruebas automatizadas de no filtración, migraciones y datos iniciales multi-inquilino, y una demo con dos operadores. El ambiente debe caber en el nivel gratuito de la nube y reconstruirse con un comando.

Decisión

Base de datos y esquema compartidos, con aislamiento por fila. Toda entidad de un operador lleva `OperadorId` y las de un comercio también `ComercioId`. El Comercio es una identidad global vinculada a cada operador mediante `CuentaComercial`. El aislamiento se aplica con filtros globales de EF Core alimentados por un `ITenantContext`, más un interceptor de `SaveChanges` que valida escrituras. El detalle de resolución por canal está en la sección 5.

Alternativas consideradas

Se evaluaron las cinco estrategias presentadas en el teórico de Arquitectura, ordenadas de más compartida a más aislada:

Estrategia	Ventajas	Desventajas	Evaluación
Compartir tablas (una base, un esquema, columna de inquilino)	Bajo costo, menos almacenamiento, una sola migración	Consultas más complejas; si una tabla se corrompe afecta a todos	Elegida. Los filtros globales de EF Core resuelven la complejidad de las consultas, y encaja con el comercio compartido entre operadores
Tablas distintas por cliente (una base)	Mejor aislamiento y personalización	Muchas más tablas, nombres distintos en cada consulta	Descartada: EF Core no está pensado para cambiar el nombre de las tablas por solicitud
Un esquema por cliente (una base)	Mejor aislamiento, las consultas son iguales	Más almacenamiento; cada migración se aplica N veces	Descartada: complejidad de migraciones, y el comercio compartido quedaría duplicado en varios esquemas
Una base por cliente (mismo motor)	Buen aislamiento, mantenimiento independiente	Más almacenamiento y memoria, límite de bases	Descartada: multiplica conexiones y costo; dar de alta un operador exigiría aprovisionar infraestructura
Un servidor por cliente	El mejor aislamiento	Más costos y licencias, límite de instancias	Descartada: excede el nivel gratuito de la nube y la escala de un proyecto de 9 semanas

Además se descartó **aplicar el aislamiento a mano** (un `WHERE` en cada consulta dentro de la opción elegida), porque depende de que nadie lo olvide y no se puede verificar de forma centralizada.

Los desafíos que el teórico asocia a multi-tenancy se atienden así: **aislamiento y seguridad** con filtros globales, interceptor de escrituras y pruebas automatizadas; **customización** con marca, tarifario y reglas configurables por operador; **actualizaciones** con una única migración para todos; y **recuperación** con respaldos de la base compartida, aceptando que no se puede restaurar un solo operador por separado.

Consecuencias

- A favor: una sola migración para todos; dar de alta un operador es insertar filas; las consultas de plataforma (métricas globales) son simples; el modelo de comercio compartido encaja naturalmente.
- En contra: un error que omita el filtro puede filtrar datos. Mitigaciones: filtros globales en lugar de `WHERE` manuales, prohibición de `IgnoreQueryFilters` fuera de un repositorio autorizado (verificada por prueba de arquitectura), pruebas automatizadas de aislamiento por endpoint, y Row-Level Security como posible defensa adicional.
- Vecino ruidoso: un operador con mucho volumen afecta el rendimiento de los demás. Se mitiga con índices compuestos que empiezan por `OperadorId` y límites de tasa por inquilino.
- Lo que dificulta a futuro: si un operador exigiera su propia base por contrato, habría que migrar sus datos a una instancia aparte; el `OperadorId` en todas las tablas facilita esa extracción, pero no es inmediato.